Лабораторная работа №3

Объектно-ориентированное программирование в Python

Вариант 20. Система обучения

1. Цель работы

Целью работы является получение практических навыков разработки программ с использованием объектно-ориентированного подхода. В ходе выполнения лабораторной работы осваиваются понятие класса и объекта, атрибуты и методы, конструктор __init__(), параметр self, свойства property, инкапсуляция, специальные методы, наследование, переопределение методов, полиморфизм, абстрактные классы и композиция объектов.

Программа должна моделировать предметную область не словарями, как в лабораторной работе №2, а объектами классов. Все данные хранятся в оперативной памяти. Сторонние библиотеки, файлы, JSON, базы данных и веб-фреймворки не используются.

2. Постановка задачи

По индивидуальному заданию варианта 20 необходимо разработать консольное приложение «Система обучения». Базовый класс LearningMaterial описывает учебный материал. Производные классы Lecture и Practice представляют лекцию и практическое занятие. Для лекции учитывается длительность, для практики — количество заданий. Общий метод evaluate_volume() оценивает объём занятия в минутах.

Класс Course содержит коллекцию учебных материалов и реализует отношение композиции Course HAS-A LearningMaterial. Программа создаёт не менее трёх экземпляров, предоставляет консольное меню и демонстрирует инкапсуляцию, наследование, полиморфизм и изменение состояния объектов.

Дополнительно реализованы третий производный класс Quiz (тест), перегрузка операторов __lt__ и __add__, поиск по нескольким параметрам, сортировка по объёму и перебор материалов курса через протокол итератора.

3. Краткие теоретические сведения

Объектно-ориентированное программирование представляет программу как набор взаимодействующих объектов. Класс задаёт структуру и поведение, объект является экземпляром класса и хранит собственное состояние. Метод __init__() вызывается при создании объекта. Параметр self обозначает текущий экземпляр.

Инкапсуляция объединяет данные и методы работы с ними и ограничивает прямой доступ к внутреннему состоянию. В Python для этого используют соглашение _attribute и свойства property с проверкой в setter. Наследование позволяет создать новый класс на основе существующего. Вызов super() обращается к родительской реализации, чаще всего к конструктору.

Полиморфизм позволяет обрабатывать объекты разных классов через общий интерфейс: достаточно вызвать один и тот же метод, а конкретная реализация выбирается по фактическому типу объекта. Абстрактный класс из модуля abc задаёт обязательный интерфейс. Композиция означает, что один объект содержит другой: это отношение HAS-A, в отличие от наследования IS-A.

4. Описание программы

Программа sistema_obucheniya.py реализует иерархию учебных материалов и контейнер-курс. LearningMaterial — абстрактный базовый класс. В нём хранятся название, тема, защищённая сложность _difficulty и статус. Сложность доступна только через property: значения вне диапазона

1…5 вызывают ValueError. Статус меняется методами complete() и reset(), отдельного setter нет.

Lecture, Practice и Quiz наследуют LearningMaterial и через super().__init__() инициализируют общую часть. У лекции индивидуальный параметр — длительность, у практики — число заданий, у теста — число вопросов. Метод evaluate_volume() для лекции возвращает длительность, для практики умножает число заданий на 15 минут, для теста — число вопросов на 2 минуты. Метод process() в каждом классе переопределён и проводит занятие по-своему.

Course хранит список материалов. Добавление, удаление, вывод, статистика и цикл for item in course: демонстрируют композицию. Метод process_all() вызывает item.process() для всех элементов коллекции без проверки типа — это полиморфизм, требуемый методичкой. Сортировка использует __lt__, сумма объёмов — оператор __add__.

Консольное меню совпадает с методичкой: создать объект, показать объекты, выполнить действие, изменить объект, удалить объект, показать статистику, выход. Программа работает в цикле while до выбора пункта 0. Ввод проверяется: пустые строки, нечисловые значения и некорректные свойства отклоняются с повторным запросом.

4.1. Структура классов

```
LearningMaterial (ABC)
      |
      |---- Lecture
      |---- Practice
      |---- Quiz          (дополнительный класс)

Course  HAS-A  список LearningMaterial

items = [Lecture(...), Practice(...), Quiz(...)]
for item in items:
    item.process()
```

4.2. Основные методы

get_info() и __str__() формируют описание объекта. evaluate_volume() оценивает объём занятия. process() проводит занятие и меняет статус на

«проведён». complete() и reset() изменяют состояние. academic_hours() для лекции переводит минуты в академические часы. Course.add(), remove_at(), process_all(), find(), sorted_by_volume() и статистика обслуживают коллекцию материалов.

5. Исходный код программы

Ниже приведён полный текст файла sistema_obucheniya.py. Точка входа — конструкция if __name__ == "__main__":.

```
"""
Лабораторная работа №3. Вариант 20.
Система обучения: объектная модель учебных материалов.
"""

from abc import ABC, abstractmethod
import sys

# Корректный вывод русских символов в консоли Windows
if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")


def read_line(prompt):
    """Читает строку. В неинтерактивном режиме повторяет ввод в консоль."""
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()


def read_non_empty(prompt):
    """Считывает непустую строку."""
    while True:
        text = read_line(prompt)
        if text:
            return text
        print("Ошибка: значение не должно быть пустым.")


def parse_int(text):
    """Преобразует строку в int или возвращает None."""
    if text.startswith("-") and text[1:].isdigit():
        return -int(text[1:])
    if text.isdigit():
        return int(text)
    return None


def read_int(prompt):
    """Считывает целое число, в том числе отрицательное."""
    while True:
        value = parse_int(read_line(prompt))
        if value is not None:
            return value
        print("Ошибка: введите целое число.")
```

```
def clip(text, width):
    """Обрезает строку до ширины колонки таблицы."""
    text = str(text)
    if len(text) <= width:
        return text
    return text[: width - 3] + "..."


# ---------------------------------------------------------------------------
# Базовый класс предметной области (п. 4.1 методички)
# ---------------------------------------------------------------------------
class LearningMaterial(ABC):
    """Учебный материал. Общий интерфейс для лекции, практики и теста."""

    # Атрибут класса: общий счётчик созданных материалов
    total_created = 0

    def __init__(self, title, topic, difficulty):
        self.title = title
        self.topic = topic
        self._status = "запланирован"
        self.difficulty = difficulty
        LearningMaterial.total_created += 1

    @staticmethod
    def check_difficulty(value):
        """Проверяет сложность. Используется свойством difficulty."""
        if isinstance(value, bool) or not isinstance(value, int):
            raise ValueError("сложность должна быть целым числом от 1 до 5")
        if value < 1 or value > 5:
            raise ValueError("сложность должна быть целым числом от 1 до 5")

    @property
    def difficulty(self):
        """Защищённый атрибут: сложность занятия (инкапсуляция, п. 4.5)."""
        return self._difficulty

    @difficulty.setter
    def difficulty(self, value):
        LearningMaterial.check_difficulty(value)
        self._difficulty = value

    @property
    def status(self):
        """Статус можно только читать: меняется методами класса."""
        return self._status

    @property
    def kind(self):
        return "материал"

    def get_info(self):
        """Человекочитаемое описание объекта."""
        return (
            f"{self.kind}: «{self.title}», тема «{self.topic}», "
            f"сложность {self.difficulty}, {self.specific_text()}, "
            f"объём {self.evaluate_volume()} мин, статус: {self.status}"
        )

    def complete(self):
        """Отмечает занятие выполненным (изменение состояния)."""
        self._status = "проведён"

    def reset(self):
        """Возвращает занятие в запланированное состояние."""
        self._status = "запланирован"

    @abstractmethod
    def evaluate_volume(self):
        """Общий метод оценки объёма занятия (минуты)."""
        pass

    @abstractmethod
    def process(self):
        """Провести занятие. У каждого типа своя реализация (полиморфизм)."""
        pass
```

```
    @abstractmethod
    def specific_text(self):
        """Краткое описание индивидуального параметра типа."""
        pass

    def __str__(self):
        return self.get_info()

    def __repr__(self):
        return f"{self.__class__.__name__}(title={self.title!r})"

    def __lt__(self, other):
        """Сравнение по объёму: нужно для сортировки (доп. задание)."""
        if not isinstance(other, LearningMaterial):
            return NotImplemented
        if self.evaluate_volume() != other.evaluate_volume():
            return self.evaluate_volume() < other.evaluate_volume()
        return self.title.lower() < other.title.lower()

    def __add__(self, other):
        """Сумма объёмов двух занятий (перегрузка оператора)."""
        if not isinstance(other, LearningMaterial):
            return NotImplemented
        return self.evaluate_volume() + other.evaluate_volume()


class Lecture(LearningMaterial):
    """Лекция. Индивидуальный параметр — длительность в минутах."""

    def __init__(self, title, topic, difficulty, duration):
        super().__init__(title, topic, difficulty)
        self.duration = duration

    @property
    def kind(self):
        return "Лекция"

    @property
    def duration(self):
        return self._duration

    @duration.setter
    def duration(self, value):
        if isinstance(value, bool) or not isinstance(value, int) or value <= 0:
            raise ValueError("длительность лекции должна быть целым числом больше 0")
        self._duration = value

    def evaluate_volume(self):
        # Для лекции объём совпадает с длительностью
        return self.duration

    def academic_hours(self):
        """Дополнительная характеристика: академические часы (45 мин)."""
        return round(self.duration / 45, 2)

    def process(self):
        print(
            f"Лекция «{self.title}» проводится {self.duration} мин "
            f"({self.academic_hours()} акад. ч)."
        )
        self.complete()
        print(f"Статус изменён: {self.status}.")

    def specific_text(self):
        return f"{self.duration} мин"


class Practice(LearningMaterial):
    """Практическое занятие. Индивидуальный параметр — число заданий."""

    MINUTES_PER_TASK = 15

    def __init__(self, title, topic, difficulty, tasks_count):
        super().__init__(title, topic, difficulty)
        self.tasks_count = tasks_count

    @property
```

```
    def kind(self):
        return "Практика"

    @property
    def tasks_count(self):
        return self._tasks_count

    @tasks_count.setter
    def tasks_count(self, value):
        if isinstance(value, bool) or not isinstance(value, int) or value <= 0:
            raise ValueError("количество заданий должно быть целым числом больше 0")
        self._tasks_count = value

    def evaluate_volume(self):
        # Каждое задание оценивается как 15 минут работы
        return self.tasks_count * Practice.MINUTES_PER_TASK

    def process(self):
        print(
            f"Практика «{self.title}»: выполняется {self.tasks_count} заданий "
            f"(оценка объёма {self.evaluate_volume()} мин)."
        )
        self.complete()
        print(f"Статус изменён: {self.status}.")

    def specific_text(self):
        return f"{self.tasks_count} заданий"


class Quiz(LearningMaterial):
    """Тест (дополнительный производный класс, п. 15 методички)."""

    MINUTES_PER_QUESTION = 2

    def __init__(self, title, topic, difficulty, questions_count):
        super().__init__(title, topic, difficulty)
        self.questions_count = questions_count

    @property
    def kind(self):
        return "Тест"

    @property
    def questions_count(self):
        return self._questions_count

    @questions_count.setter
    def questions_count(self, value):
        if isinstance(value, bool) or not isinstance(value, int) or value <= 0:
            raise ValueError("количество вопросов должно быть целым числом больше 0")
        self._questions_count = value

    def evaluate_volume(self):
        return self.questions_count * Quiz.MINUTES_PER_QUESTION

    def process(self):
        print(
            f"Тест «{self.title}»: {self.questions_count} вопросов "
            f"(на выполнение {self.evaluate_volume()} мин)."
        )
        self.complete()
        print(f"Статус изменён: {self.status}.")

    def specific_text(self):
        return f"{self.questions_count} вопросов"


class Course:
    """
    Курс содержит учебные материалы.
    Композиция: Course HAS-A LearningMaterial (п. 4.8, п. 9 методички).
    """

    def __init__(self, name, teacher):
        self.name = name
        self.teacher = teacher
        self._materials = []
```

```
    def add(self, material):
        self._materials.append(material)

    def remove_at(self, index):
        return self._materials.pop(index)

    def get(self, index):
        return self._materials[index]

    def is_empty(self):
        return len(self._materials) == 0

    def process_all(self):
        """Полиморфная обработка коллекции разных типов (п. 4.7, п. 8)."""
        print("\nПолиморфный вызов process() для всех материалов:")
        for item in self._materials:
            item.process()

    def volumes(self):
        total = 0
        for item in self._materials:
            total += item.evaluate_volume()
        return total

    def find(self, topic="", kind="", min_difficulty=None):
        """Поиск по нескольким параметрам (доп. задание)."""
        result = []
        topic_lower = topic.lower()
        kind_lower = kind.lower()
        for item in self._materials:
            if topic_lower and topic_lower not in item.topic.lower():
                continue
            if kind_lower and kind_lower not in item.kind.lower():
                continue
            if min_difficulty is not None and item.difficulty < min_difficulty:
                continue
            result.append(item)
        return result

    def sorted_by_volume(self):
        """Сортировка за счёт перегрузки __lt__."""
        return sorted(self._materials)

    def count_by_kind(self):
        counts = {}
        for item in self._materials:
            counts[item.kind] = counts.get(item.kind, 0) + 1
        return counts

    def count_completed(self):
        done = 0
        for item in self._materials:
            if item.status == "проведён":
                done += 1
        return done

    def __iter__(self):
        """Собственный перебор материалов курса (доп. задание)."""
        return iter(self._materials)

    def __len__(self):
        return len(self._materials)

    def __str__(self):
        return (
            f"Курс «{self.name}», преподаватель {self.teacher}, "
            f"материалов: {len(self._materials)}"
        )


def read_difficulty():
    """Считывает сложность через ту же проверку, что и property."""
    while True:
        value = read_int("  Сложность (1-5): ")
        try:
            LearningMaterial.check_difficulty(value)
```

```
            return value
        except ValueError as exc:
            print(f"Ошибка: {exc}")


def read_positive_field(prompt, error_text):
    """Считывает целое число больше нуля."""
    while True:
        value = read_int(prompt)
        if value > 0:
            return value
        print(f"Ошибка: {error_text}")


def create_lecture():
    title = read_non_empty("  Название лекции: ")
    topic = read_non_empty("  Тема: ")
    difficulty = read_difficulty()
    duration = read_positive_field(
        "  Длительность, мин: ",
        "длительность должна быть больше нуля.",
    )
    return Lecture(title, topic, difficulty, duration)


def create_practice():
    title = read_non_empty("  Название практики: ")
    topic = read_non_empty("  Тема: ")
    difficulty = read_difficulty()
    tasks = read_positive_field(
        "  Количество заданий: ",
        "количество заданий должно быть больше нуля.",
    )
    return Practice(title, topic, difficulty, tasks)


def create_quiz():
    title = read_non_empty("  Название теста: ")
    topic = read_non_empty("  Тема: ")
    difficulty = read_difficulty()
    questions = read_positive_field(
        "  Количество вопросов: ",
        "количество вопросов должно быть больше нуля.",
    )
    return Quiz(title, topic, difficulty, questions)


def print_materials(materials, title="Список учебных материалов"):
    print(f"\n{title}:")
    if len(materials) == 0:
        print("Подходящих объектов нет.")
        return

    print(
        f"{'N':<4}{'Тип':<12}{'Название':<28}{'Тема':<16}"
        f"{'Сл.':<5}{'Параметр':<16}{'Объём':<8}{'Статус'}"
    )
    for index, item in enumerate(materials, start=1):
        print(
            f"{index:<4}"
            f"{clip(item.kind, 11):<12}"
            f"{clip(item.title, 27):<28}"
            f"{clip(item.topic, 15):<16}"
            f"{item.difficulty:<5}"
            f"{clip(item.specific_text(), 15):<16}"
            f"{item.evaluate_volume():<8}"
            f"{item.status}"
        )


def choose_index(course, prompt="Введите номер объекта: "):
    while True:
        value = read_int(prompt)
        if 1 <= value <= len(course):
            return value - 1
        print(f"Ошибка: номер должен быть от 1 до {len(course)}.")
```

```
def ensure_not_empty(course):
    if course.is_empty():
        print("Ошибка: сначала создайте хотя бы один объект.")
        return False
    return True


def create_object(course):
    print("\nТип учебного материала:")
    print("  1. Лекция")
    print("  2. Практика")
    print("  3. Тест")
    choice = read_line("Выберите тип: ")

    if choice == "1":
        material = create_lecture()
    elif choice == "2":
        material = create_practice()
    elif choice == "3":
        material = create_quiz()
    else:
        print("Ошибка: выберите тип 1, 2 или 3.")
        return

    course.add(material)
    print("Объект создан.")
    print(material)


def run_action(course):
    print("\nДействие:")
    print("  1. Провести выбранное занятие (process)")
    print("  2. Оценить объём выбранного занятия")
    print("  3. Провести все занятия (полиморфизм)")
    print("  4. Поиск по нескольким параметрам")
    print("  5. Сортировка по объёму")
    print("  6. Суммарный объём двух занятий (оператор +)")
    choice = read_line("Выберите действие: ")

    if choice == "1":
        print_materials(list(course))
        item = course.get(choose_index(course))
        print()
        item.process()

    elif choice == "2":
        print_materials(list(course))
        item = course.get(choose_index(course))
        print(f"\nОценка объёма «{item.title}»: {item.evaluate_volume()} мин")
        print(item.get_info())

    elif choice == "3":
        course.process_all()

    elif choice == "4":
        print("Пустое значение означает «любой».")
        topic = read_line("  Тема (фрагмент): ")
        kind = read_line("  Тип (Лекция/Практика/Тест): ")
        min_text = read_line("  Минимальная сложность (Enter — любая): ")
        min_difficulty = None
        if min_text:
            min_difficulty = parse_int(min_text)
            if min_difficulty is None:
                print("Ошибка: минимальная сложность должна быть целым числом.")
                return
        found = course.find(topic, kind, min_difficulty)
        print_materials(found, "Результат поиска")

    elif choice == "5":
        ordered = course.sorted_by_volume()
        print_materials(ordered, "Материалы по возрастанию объёма")

    elif choice == "6":
        if len(course) < 2:
            print("Ошибка: нужно минимум два объекта.")
            return
```

```
        print_materials(list(course))
        first = course.get(choose_index(course, "Номер первого объекта: "))
        second = course.get(choose_index(course, "Номер второго объекта: "))
        total = first + second
        print(
            f"\n{first.kind} «{first.title}» + {second.kind} «{second.title}» "
            f"= {total} мин"
        )

    else:
        print("Ошибка: выберите пункт от 1 до 6.")


def edit_object(course):
    print_materials(list(course))
    item = course.get(choose_index(course))
    print("\nТекущее состояние:")
    print(item)
    print("\nЧто изменить?")
    print("  1. Название")
    print("  2. Тема")
    print("  3. Сложность")
    print("  4. Индивидуальный параметр")
    print("  5. Сбросить статус в «запланирован»")
    choice = read_line("Выберите поле: ")

    if choice == "1":
        item.title = read_non_empty("Новое название: ")
    elif choice == "2":
        item.topic = read_non_empty("Новая тема: ")
    elif choice == "3":
        while True:
            value = read_int("Новая сложность (1-5): ")
            try:
                item.difficulty = value
                break
            except ValueError as exc:
                print(f"Ошибка: {exc}")
    elif choice == "4":
        change_specific_parameter(item)
    elif choice == "5":
        item.reset()
    else:
        print("Ошибка: выберите пункт от 1 до 5.")
        return

    print("Объект изменён.")
    print(item)


def change_specific_parameter(item):
    """Изменяет длительность, число заданий или число вопросов."""
    while True:
        try:
            if isinstance(item, Lecture):
                item.duration = read_int("Новая длительность, мин: ")
            elif isinstance(item, Practice):
                item.tasks_count = read_int("Новое количество заданий: ")
            elif isinstance(item, Quiz):
                item.questions_count = read_int("Новое количество вопросов: ")
            return
        except ValueError as exc:
            print(f"Ошибка: {exc}")


def delete_object(course):
    print_materials(list(course))
    index = choose_index(course)
    item = course.get(index)
    print("\nОбъект будет удалён:")
    print(item)
    confirm = read_line("Подтвердите удаление (да/нет): ")
    if confirm.lower() in ("да", "д", "yes", "y"):
        course.remove_at(index)
        print("Объект удалён.")
    else:
        print("Удаление отменено.")
```

```
def print_statistics(course):
    print("\n----- Статистика курса -----")
    print(course)
    print(f"Всего создано материалов за сессию: {LearningMaterial.total_created}")
    print(f"Сейчас в курсе: {len(course)}")
    if course.is_empty():
        return

    print("По типам:")
    for kind, count in course.count_by_kind().items():
        print(f"  {kind}: {count}")

    total_volume = course.volumes()
    done = course.count_completed()
    print(f"Суммарный объём занятий: {total_volume} мин")
    print(f"Проведено: {done}, запланировано: {len(course) - done}")

    hardest = course.get(0)
    for item in course:
        if item.difficulty > hardest.difficulty:
            hardest = item
        elif item.difficulty == hardest.difficulty and item.evaluate_volume() > hardest.evaluate_volume():
            hardest = item
    print(f"Самый сложный материал: {hardest.kind} «{hardest.title}» (сложность {hardest.difficulty})")

    ordered = course.sorted_by_volume()
    print(f"Наименьший объём: {ordered[0].kind} «{ordered[0].title}» — {ordered[0].evaluate_volume()}
мин")
    print(f"Наибольший объём: {ordered[-1].kind} «{ordered[-1].title}» — {ordered[-1].evaluate_volume()}
мин")


def print_menu():
    print()
    print("===== МЕНЮ =====")
    print()
    print("1. Создать объект")
    print("2. Показать объекты")
    print("3. Выполнить действие")
    print("4. Изменить объект")
    print("5. Удалить объект")
    print("6. Показать статистику")
    print("0. Выход")
    print()


def main():
    print("Лабораторная работа №3. Вариант 20 — система обучения")
    print("Курс содержит учебные материалы (композиция Course HAS-A LearningMaterial).")
    print()
    name = read_non_empty("Название курса: ")
    teacher = read_non_empty("Преподаватель: ")
    course = Course(name, teacher)
    print(f"Создан объект курса: {course}")

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")

        if choice == "1":
            create_object(course)
        elif choice == "2":
            if ensure_not_empty(course):
                print_materials(list(course), "Все объекты курса")
                print("\nПодробно:")
                for item in course:
                    print(" -", item)
        elif choice == "3":
            if ensure_not_empty(course):
                run_action(course)
        elif choice == "4":
            if ensure_not_empty(course):
                edit_object(course)
        elif choice == "5":
            if ensure_not_empty(course):
```

```
                delete_object(course)
        elif choice == "6":
            print_statistics(course)
        elif choice == "0":
            print("Работа программы завершена.")
            break
        else:
            print("Ошибка: выберите пункт меню от 0 до 6.")


if __name__ == "__main__":
    main()
```

6. Результаты работы программы

Ниже приведены снимки консольного сеанса. Сначала создаётся объект курса (композиция), затем через меню добавляются материалы разных типов, выполняются действия, изменяется состояние и выводится статистика.

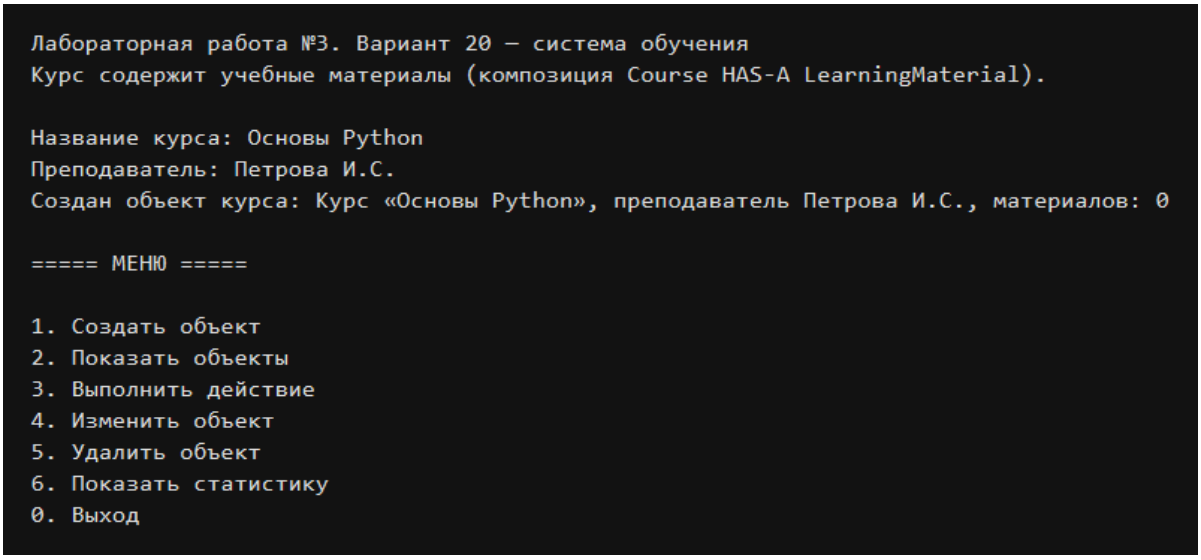


Рисунок 1 – Создание курса и консольное меню программы

```
Выберите действие: 1

Тип учебного материала:
  1. Лекция
  2. Практика
  3. Тест
Выберите тип: 1
  Название лекции:
Ошибка: значение не должно быть пустым.
  Название лекции: Введение в ООП
  Тема: ООП
  Сложность (1-5): 0
Ошибка: сложность должна быть целым числом от 1 до 5
  Сложность (1-5): 6
Ошибка: сложность должна быть целым числом от 1 до 5
  Сложность (1-5): abc
Ошибка: введите целое число.
  Сложность (1-5): 2
  Длительность, мин: -10
Ошибка: длительность должна быть больше нуля.
  Длительность, мин: 0
Ошибка: длительность должна быть больше нуля.
  Длительность, мин: 90
Объект создан.
Лекция: «Введение в ООП», тема «ООП», сложность 2, 90 мин, объём 90 мин, статус: запланирован
```

Рисунок 2 – Создание лекции и отсечение некорректных значений свойства

```
Выберите действие: 1

Тип учебного материала:
  1. Лекция
  2. Практика
  3. Тест
Выберите тип: 2
  Название практики: Практикум: классы и объекты
  Тема: ООП
  Сложность (1-5): 3
  Количество заданий: 0
Ошибка: количество заданий должно быть больше нуля.
  Количество заданий: 8
Объект создан.
Практика: «Практикум: классы и объекты», тема «ООП», сложность 3, 8 заданий, объём 120 мин, статус: запланирован
Выберите действие: 1

Тип учебного материала:
  1. Лекция
  2. Практика
  3. Тест
Выберите тип: 3
  Название теста: Проверка знаний по ООП
  Тема: ООП
  Сложность (1-5): 4
  Количество вопросов: 20
Объект создан.
Тест: «Проверка знаний по ООП», тема «ООП», сложность 4, 20 вопросов, объём 40 мин, статус: запланирован
Выберите действие: 1

Тип учебного материала:
  1. Лекция
  2. Практика
  3. Тест
Выберите тип: 1
  Название лекции: Наследование в Python
  Тема: Наследование
  Сложность (1-5): 5
  Длительность, мин: 45
Объект создан.
Лекция: «Наследование в Python», тема «Наследование», сложность 5, 45 мин, объём 45 мин, статус: запланирован
```

Рисунок 3 – Создание практики, теста и второй лекции

```
Выберите действие: 2

Все объекты курса:
N   Тип         Название                  Тема           Сл.  Параметр       Объём   Статус
1   Лекция      Введение в ООП            ООП            2    90 мин         90      запланирован
2   Практика    Практикум: классы и объекты ООП          3    8 заданий      120     запланирован
3   Тест        Проверка знаний по ООП    ООП            4    20 вопросов    40      запланирован
4   Лекция      Наследование в Python     Наследование   5    45 мин         45      запланирован

Подробно:
 - Лекция: «Введение в ООП», тема «ООП», сложность 2, 90 мин, объём 90 мин, статус: запланирован
 - Практика: «Практикум: классы и объекты», тема «ООП», сложность 3, 8 заданий, объём 120 мин, статус: запланирован
 - Тест: «Проверка знаний по ООП», тема «ООП», сложность 4, 20 вопросов, объём 40 мин, статус: запланирован
 - Лекция: «Наследование в Python», тема «Наследование», сложность 5, 45 мин, объём 45 мин, статус: запланирован
```

Рисунок 4 – Вывод всех объектов курса

```
Выберите действие: 3

Действие:
  1. Провести выбранное занятие (process)
  2. Оценить объём выбранного занятия
  3. Провести все занятия (полиморфизм)
  4. Поиск по нескольким параметрам
  5. Сортировка по объёму
  6. Суммарный объём двух занятий (оператор +)
Выберите действие: 2

Список учебных материалов:
N   Тип         Название                  Тема           Сл.  Параметр       Объём   Статус
1   Лекция      Введение в ООП            ООП            2    90 мин         90      запланирован
2   Практика    Практикум: классы и объекты ООП          3    8 заданий      120     запланирован
3   Тест        Проверка знаний по ООП    ООП            4    20 вопросов    40      запланирован
4   Лекция      Наследование в Python     Наследование   5    45 мин         45      запланирован
Введите номер объекта: 1

Оценка объёма «Введение в ООП»: 90 мин
Лекция: «Введение в ООП», тема «ООП», сложность 2, 90 мин, объём 90 мин, статус: запланирован
```

Рисунок 5 – Общий метод оценки объёма занятия

```
Выберите действие: 3

Действие:
  1. Провести выбранное занятие (process)
  2. Оценить объём выбранного занятия
  3. Провести все занятия (полиморфизм)
  4. Поиск по нескольким параметрам
  5. Сортировка по объёму
  6. Суммарный объём двух занятий (оператор +)
Выберите действие: 3

Полиморфный вызов process() для всех материалов:
Лекция «Введение в ООП» проводится 90 мин (2.0 акад. ч).
Статус изменён: проведён.
Практика «Практикум: классы и объекты»: выполняется 8 заданий (оценка объёма 120 мин).
Статус изменён: проведён.
Тест «Проверка знаний по ООП»: 20 вопросов (на выполнение 40 мин).
Статус изменён: проведён.
Лекция «Наследование в Python» проводится 45 мин (1.0 акад. ч).
Статус изменён: проведён.
```

Рисунок 6 – Полиморфный вызов process() для коллекции разных типов

```
Введите номер объекта: 1

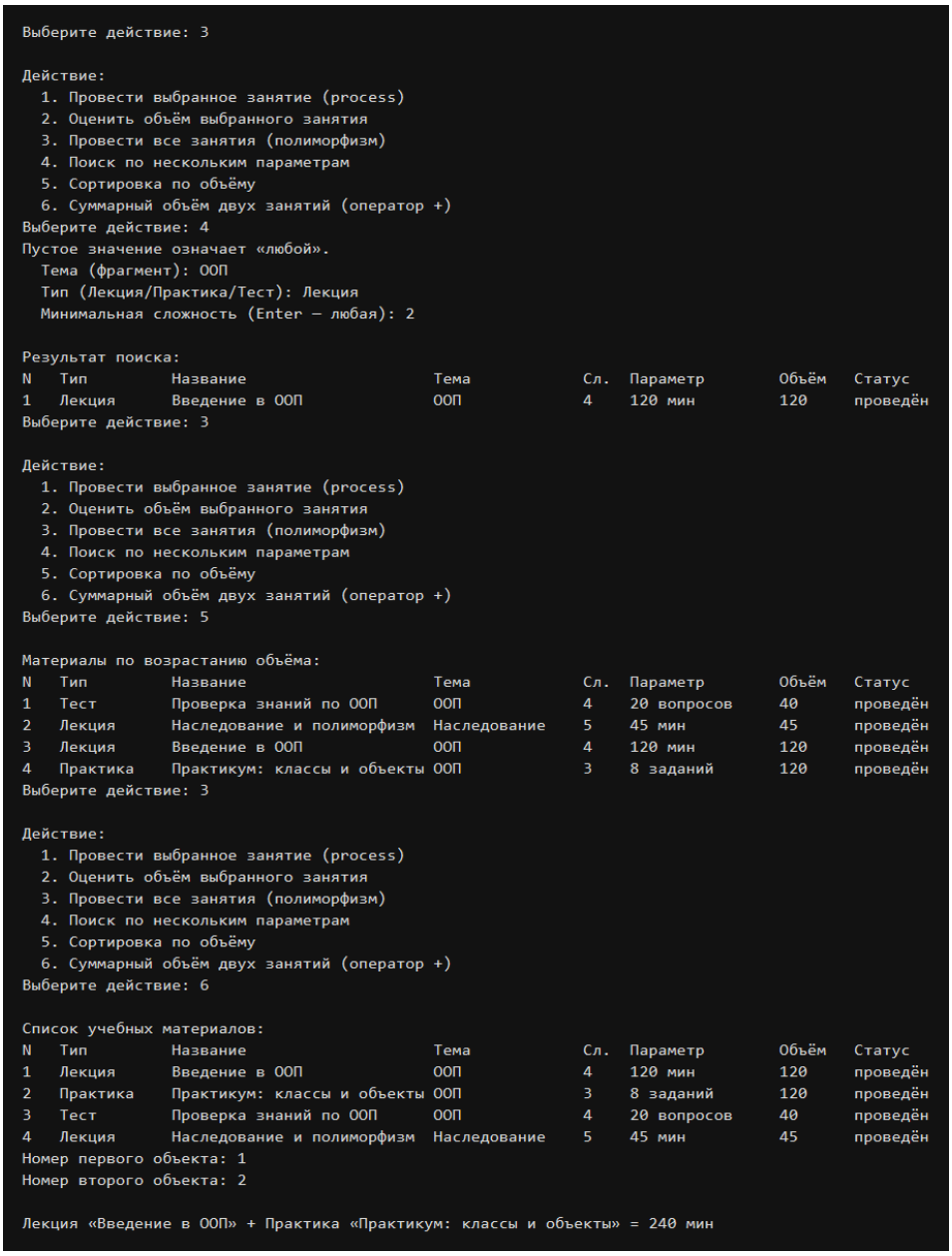
Текущее состояние:
Лекция: «Введение в ООП», тема «ООП», сложность 2, 90 мин, объём 90 мин, статус: проведён

Что изменить?
  1. Название
  2. Тема
  3. Сложность
  4. Индивидуальный параметр
  5. Сбросить статус в «запланирован»
Выберите поле: 3
Новая сложность (1-5): -1
Ошибка: сложность должна быть целым числом от 1 до 5
Новая сложность (1-5): 7
Ошибка: сложность должна быть целым числом от 1 до 5
Новая сложность (1-5): 4
Объект изменён.
Лекция: «Введение в ООП», тема «ООП», сложность 4, 90 мин, объём 90 мин, статус: проведён
Введите номер объекта: 1

Текущее состояние:
Лекция: «Введение в ООП», тема «ООП», сложность 4, 90 мин, объём 90 мин, статус: проведён

Что изменить?
  1. Название
  2. Тема
  3. Сложность
  4. Индивидуальный параметр
  5. Сбросить статус в «запланирован»
Выберите поле: 4
Новая длительность, мин: -5
Ошибка: длительность лекции должна быть целым числом больше 0
Новая длительность, мин: 0
Ошибка: длительность лекции должна быть целым числом больше 0
Новая длительность, мин: 120
Объект изменён.
Лекция: «Введение в ООП», тема «ООП», сложность 4, 120 мин, объём 120 мин, статус: проведён
```

Рисунок 7 – Изменение состояния объекта и обработка некорректного значения property



```
Выберите действие: 3

Действие:
  1. Провести выбранное занятие (process)
  2. Оценить объём выбранного занятия
  3. Провести все занятия (полиморфизм)
  4. Поиск по нескольким параметрам
  5. Сортировка по объёму
  6. Суммарный объём двух занятий (оператор +)
Выберите действие: 4
Пустое значение означает «любой».
  Тема (фрагмент): ООП
  Тип (Лекция/Практика/Тест): Лекция
  Минимальная сложность (Enter — любая): 2

Результат поиска:
N   Тип         Название                    Тема           Сл.  Параметр       Объём   Статус
1   Лекция      Введение в ООП              ООП            4    120 мин        120     проведён
Выберите действие: 3

Действие:
  1. Провести выбранное занятие (process)
  2. Оценить объём выбранного занятия
  3. Провести все занятия (полиморфизм)
  4. Поиск по нескольким параметрам
  5. Сортировка по объёму
  6. Суммарный объём двух занятий (оператор +)
Выберите действие: 5

Материалы по возрастанию объёма:
N   Тип         Название                    Тема           Сл.  Параметр       Объём   Статус
1   Тест        Проверка знаний по ООП      ООП            4    20 вопросов    40      проведён
2   Лекция      Наследование и полиморфизм  Наследование   5    45 мин         45      проведён
3   Лекция      Введение в ООП              ООП            4    120 мин        120     проведён
4   Практика    Практикум: классы и объекты ООП            3    8 заданий      120     проведён
Выберите действие: 3

Действие:
  1. Провести выбранное занятие (process)
  2. Оценить объём выбранного занятия
  3. Провести все занятия (полиморфизм)
  4. Поиск по нескольким параметрам
  5. Сортировка по объёму
  6. Суммарный объём двух занятий (оператор +)
Выберите действие: 6

Список учебных материалов:
N   Тип         Название                    Тема           Сл.  Параметр       Объём   Статус
1   Лекция      Введение в ООП              ООП            4    120 мин        120     проведён
2   Практика    Практикум: классы и объекты ООП            3    8 заданий      120     проведён
3   Тест        Проверка знаний по ООП      ООП            4    20 вопросов    40      проведён
4   Лекция      Наследование и полиморфизм  Наследование   5    45 мин         45      проведён
Номер первого объекта: 1
Номер второго объекта: 2

Лекция «Введение в ООП» + Практика «Практикум: классы и объекты» = 240 мин
```

Рисунок 8 – Поиск по нескольким параметрам, сортировка и перегрузка оператора +

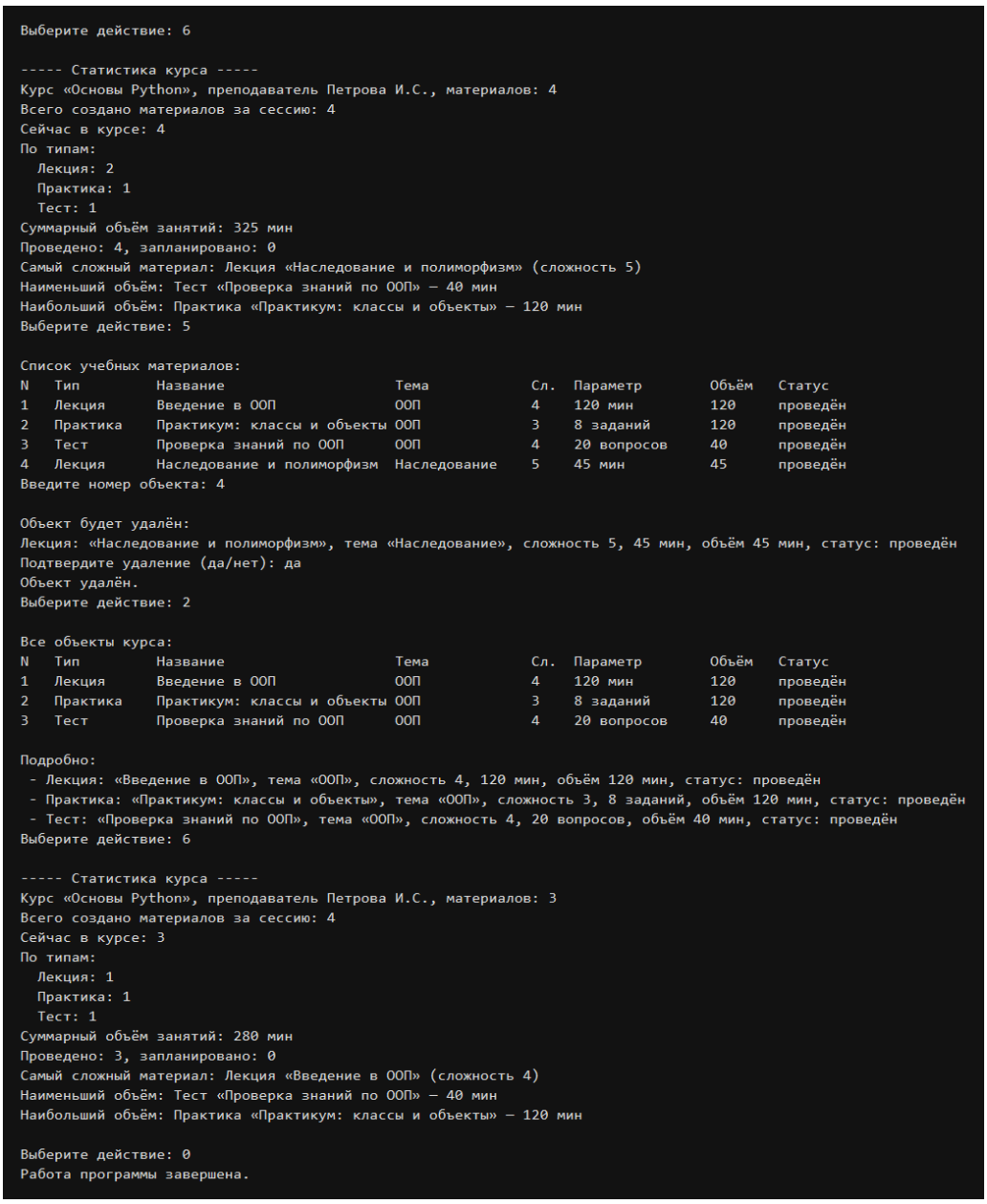


Рисунок 9 – Статистика курса, удаление объекта и завершение работы

7. Результаты тестирования

Тестирование выполнено в соответствии с пунктом 12 методических указаний. Все данные вводились через консольное меню.

Тест 1. Создание объектов каждого типа.

Через пункт меню 1 созданы Lecture «Введение в ООП», Practice «Практикум: классы и объекты», Quiz «Проверка знаний по ООП» и ещё одна лекция «Наследование в Python». Объектов больше трёх, присутствуют

оба обязательных производных класса и дополнительный класс Quiz. Результат: пройден (рисунки 2–4).

Тест 2. Корректное выполнение общих методов.

Для созданных объектов вызываются get_info()/__str__(), вывод таблицы и evaluate_volume(). Для лекции объём равен 90 мин, для практики 8×15=120 мин, для теста 20×2=40 мин. Результат: пройден (рисунки 4 и 5).

Тест 3. Работа переопределённых методов.

Метод process() в каждом классе имеет свою реализацию: лекция сообщает длительность и академические часы, практика — число заданий, тест — число вопросов. Результат: пройден (рисунок 6).

Тест 4. Изменение состояния объекта.

После process() статус меняется с «запланирован» на «проведён». Через пункт «Изменить объект» сложность лекции изменена с 2 на 4, длительность — с 90 на 120 мин, название второй лекции заменено. Результат: пройден (рисунки 6 и 7).

Тест 5. Обработка некорректного значения свойства.

При создании и изменении объекта значения сложности 0, 6, −1, 7 и длительности −10, 0, −5 отклоняются свойством: выводится ValueError в пользовательской форме, объект не получает недопустимое состояние. Результат: пройден (рисунки 2 и 7).

Тест 6. Работа полиморфной коллекции.

Курс содержит объекты Lecture, Practice и Quiz. Цикл for item in self._materials: item.process() обрабатывает все элементы одним вызовом без isinstance. Каждый объект выполняет свою версию метода. Результат: пройден (рисунок 6).

Тест 7. Работа композиции объектов.

Объект Course «Основы Python» хранит список материалов. Статистика показывает число элементов в курсе, суммарный объём и распределение по типам. После удаления четвёртого объекта в курсе остаётся 3 материала, счётчик созданных за сессию по-прежнему равен 4. Результат: пройден (рисунок 9).

8. Контрольные вопросы

1. Что такое объектно-ориентированное программирование?

ООП — подход к разработке, при котором программа представляется как совокупность взаимодействующих объектов. Объект объединяет данные (состояние) и методы (поведение). В данной работе объекты — учебные материалы и курс, который ими управляет.

2. Что такое класс?

Класс — описание структуры и поведения объектов, «чертёж». По классу LearningMaterial задаются общие поля и методы всех учебных материалов, хотя сам абстрактный класс не создаётся.

3. Что такое объект?

Объект — конкретный экземпляр класса, обладающий собственным состоянием. Например, Lecture("Введение в ООП", "ООП", 2, 90) — отдельный объект лекции с названием, темой, сложностью и длительностью.

4. Чем класс отличается от объекта?

Класс описывает тип, объект — его конкретный экземпляр. У одного класса может быть много объектов. Класс Lecture один, а лекций в курсе может быть несколько, и изменение одной не меняет другую.

5. Для чего используется __init__()?

Метод __init__() — конструктор. Он вызывается при создании объекта и задаёт начальное состояние: название, тему, сложность и индивидуальный параметр типа.

6. Что означает параметр self?

self — ссылка на конкретный экземпляр, для которого вызван метод. Через self.title и self._difficulty метод работает с данными именно этого объекта, а не другого.

7. Что такое атрибут объекта?

Атрибут объекта хранит часть его состояния. У материала это title, topic, _difficulty, _status; у лекции дополнительно _duration.

8. Что такое метод?

Метод — функция, описанная в классе и определяющая поведение объекта. Примеры: process(), evaluate_volume(), get_info(), add() у курса.

9. Чем атрибут класса отличается от атрибута экземпляра?

Атрибут класса общий для всех экземпляров, если экземпляр не создаёт собственное одноимённое поле. LearningMaterial.total_created считает все созданные материалы. Атрибут экземпляра self.title принадлежит только одному объекту.

10. Что такое инкапсуляция?

Инкапсуляция — объединение данных и методов внутри объекта и ограничение неконтролируемого доступа к внутреннему состоянию. Сложность и длительность изменяются через property, статус — только методами complete() и reset().

11. Что означает _attribute в Python?

Одинарное подчёркивание — соглашение о внутреннем атрибуте. Поле _difficulty не предназначено для прямого обращения снаружи класса.

Двойное подчёркивание запускает name mangling, но жёсткого запрета доступа в Python нет.

12. Для чего используется property?

Свойство позволяет обращаться к полю как к атрибуту (material.difficulty = 4), но при чтении и записи вызываются методы. Так программа контролирует допустимость значения.

13. Зачем нужен setter?

Setter задаёт новое значение после проверки. Если сложность меньше 1 или больше 5, setter возбуждает ValueError, и состояние объекта не портится.

14. Что такое наследование?

Наследование позволяет создать новый класс на основе существующего. Lecture(LearningMaterial) получает общие поля и методы и добавляет длительность, а также свою реализацию evaluate_volume() и process().

15. Для чего используется super()?

super() вызывает реализацию родительского класса. В конструкторах Lecture, Practice и Quiz используется super().__init__(title, topic, difficulty), чтобы не дублировать инициализацию общей части.

16. Что такое переопределение метода?

Производный класс заменяет реализацию метода родителя своей. process() и evaluate_volume() объявлены в базе как абстрактные и по-разному реализованы в Lecture, Practice и Quiz.

17. Что такое полиморфизм?

Полиморфизм позволяет работать с объектами разных классов через общий интерфейс. Для курса достаточно вызвать item.process() или item.evaluate_volume(), не выясняя, лекция это, практика или тест.

18. В чём заключается преимущество полиморфизма?

Не нужно писать цепочки if isinstance(...). Новый производный класс достаточно подключить к той же коллекции — цикл process_all() продолжит работать без переделки.

19. Что такое абстрактный класс?

Абстрактный класс задаёт общий интерфейс группы классов и сам по себе неполон. LearningMaterial нельзя использовать как готовый материал: у него нет собственной формулы объёма.

20. Для чего используется модуль abc?

Модуль abc (Abstract Base Classes) даёт класс ABC и декоратор @abstractmethod. С их помощью базовый учебный материал объявлен абстрактным, а evaluate_volume() и process() — обязательными для наследников.

21. Что такое абстрактный метод?

Это метод без реализации в базовом классе. Наследник обязан его определить, иначе его тоже нельзя создать. В программе так объявлены evaluate_volume(), process() и specific_text().

22. Что такое композиция?

Композиция означает, что один объект содержит другой. Course хранит список объектов LearningMaterial и вызывает их методы. Уничтожение или очистка курса затрагивает эту коллекцию.

23. Чем композиция отличается от наследования?

Наследование описывает отношение «является» (IS-A), композиция — «содержит» (HAS-A). Лекция является учебным материалом, курс содержит учебные материалы, но сам материалом не является.

24. Что означает отношение IS-A?

Это отношение наследования: объект производного класса является объектом базового. Lecture IS-A LearningMaterial, поэтому лекцию можно хранить в списке материалов и вызывать общие методы.

25. Что означает отношение HAS-A?

Это отношение композиции: объект включает в себя другой объект. Course HAS-A LearningMaterial: у курса есть поле-коллекция материалов.

26. Для чего используется __str__()?

Специальный метод __str__() задаёт человекочитаемое представление объекта. Функция print(material) выводит результат get_info(), а не служебный адрес объекта в памяти.

27. Что такое специальный метод Python?

Это метод с именем из двойных подчёркиваний, который вызывается интерпретатором в особых ситуациях: __init__ при создании, __str__ при печати, __lt__ при сортировке, __add__ при операции +, __iter__ при for.

28. Можно ли создать объект абстрактного класса?

Нет. Попытка LearningMaterial(...) приводит к ошибке, пока не реализованы все абстрактные методы. Создаются только Lecture, Practice и Quiz.

29. Почему список объектов разных классов может быть обработан одним циклом?

Потому что все они поддерживают один интерфейс базового класса. Цикл вызывает process() или evaluate_volume(), а интерпретатор сам выбирает реализацию по фактическому типу объекта.

30. Какие преимущества ООП даёт по сравнению с программой на функциях и коллекциях?

Данные и операции над ними живут вместе, состояние защищено свойствами, поведение расширяется наследованием, а полиморфизм убирает

разрозненные проверки типа. В лабораторной работе №2 заказ был словарём, и правила были «снаружи». Здесь материал сам знает, как оценить объём и как провести занятие, а курс только хранит и координирует объекты.

9. Вывод

В ходе лабораторной работы разработана объектная модель системы обучения. Реализованы базовый абстрактный класс LearningMaterial, производные классы Lecture и Practice, дополнительный класс Quiz и контейнер Course. Программа демонстрирует инкапсуляцию через property, наследование и super(), переопределение методов, полиморфную обработку коллекции и композицию Course HAS-A LearningMaterial.

Консольное меню соответствует методичке, ввод проверяется, некорректные значения свойств отклоняются. Дополнительно выполнены перегрузка операторов, поиск по нескольким параметрам, сортировка и итерация по курсу. Все семь обязательных тестов пройдены. Поставленная цель работы достигнута.